

Obstacle Detection Using Image Processing Techniques

Omar Mones*, Amr Sayed*,

Zakaria Shamseldin Eleila*, Karim Hassan Ali *, Mohammed Hany * and AbdElRahman Mahmoud Refaat Gad*

**German University in Cairo (GUC), Egypt*

Emails: omar.hendawy@student.guc.edu.eg, amr.behery@student.guc.edu.eg, zakaria.eleila@student.guc.edu.eg, Karim.ali@student.guc.edu.eg, mohammed.hany@student.guc.edu.eg, abdelrahman.gad@student.guc.edu.eg

Abstract—This report presents the design and implementation of an obstacle detection and avoidance system for an autonomous vehicle using image processing techniques. The system utilizes a Raspberry Pi 5 paired with a camera module to detect obstacles represented by a QR-coded cube. Image frames are processed through grayscale conversion, histogram equalization, and binary thresholding to identify the characteristic features of a QR code. Upon detection, the system estimates the distance to the obstacle using a geometric relationship between real-world dimensions and image measurements. The architecture integrates multiple modules, including camera processing, localization using an Extended Kalman Filter (EKF), obstacle avoidance logic, and a Stanley controller for path tracking. Experimental results demonstrate reliable detection within a range of up to 60 cm, with an average detection time of 0.342 seconds. The system successfully performs lane-switching maneuvers to avoid obstacles, validating its effectiveness for real-time autonomous navigation applications.

Keywords—Obstacle Detection; QR code; Stanley; EKF.

I. INTRODUCTION

Autonomous vehicles have attracted significant attention over the past two decades, with extensive research devoted to addressing the many challenges facing the autonomous driving industry. Among these challenges, perception remains one of the most critical, as it concerns how a vehicle interprets and understands its surrounding environment. A key aspect of perception is obstacle detection.

This project aims to design and implement a computer vision system using a Raspberry Pi 5 and a 5 MP camera module to detect a cube representing an obstacle with dimensions of 5×5 cm. The system will employ image processing techniques to identify the cube using a QR code that is stuck onto it. In addition to detection, the system will estimate the cube's position within the image frame and approximate its distance from the camera. The ultimate objective is to develop a reliable real-time detection framework that can later be integrated with robotic control or autonomous navigation systems.

This paper is divided as follows: Section II shows previous literature on this topic, section III shows the used hardware system and the circuit diagram, section IV will explain the overall system algorithm logic, section V shows the simulations done on the system and comparing it to real

life testing and section VI shows the concluding thoughts on the results of the project.

II. LITERATURE REVIEW

The topic of obstacle detection using image processing techniques has been tackled by different papers. In [1], lanes, street signs and obstacles were detected using a multistage pre-processing technique. The images were then further processed where Canny edge detection and Hough transform techniques were used to detect the required objects. In [2], a Water Obstacle Detection network based on Image Segmentation (WODIS) was developed where it utilized an encoder network, an Attention Refine Module (ARM) and a Feature Fusion Module (FFM). In [3], a depth camera is used for omni-directional obstacle detection. A method applying region growth for depth image and a fast inpainting method for depth image was proposed to extract and repair the obstacle regions in the depth images. After that, an improved method of iterative normalized cut was used to cluster and segment fragmentary and irregular obstacle regions to generate the complete obstacle regions which are viewed using a three-dimensional visualization method. In [4], a reconstruction module that can be used with many existing semantic segmentation networks and that is trained to recognize and reconstruct the drivable surface from a small bottleneck was proposed where any poor reconstruction of the road surface was attributed to the existence of obstacles in the vehicle's way. In [5], two different optimization techniques (Ant Colony Optimization (ACO) and Bee Colony Optimization (BCO) algorithms) were used for edge identification to help in obstacle detection.

III. HARDWARE DESIGN AND IMPLEMENTATION

In this section, the hardware components, in addition to the circuit diagram are going to be shown.

A. Vehicle Assembly

B. Hardware Components

Table I shows the components used to build the used vehicle.

C. Circuit Diagram

Figure 2 shows the circuit diagram used in our vehicle.

Table I: Hardware Components Table

Component	Quantity	Price(EGP)	Where to buy
Raspberry PI 5 Model B 8GB	1	6300	Most Electronics
ESP32	1	412	Amazon
Power Bank	1	2800	Xiaomistore
Micro-Servo Motor	1	90	Makers Electronics
DC Gear Motor With Encoder	1	475	Ampere Electronics
Jumper Wires	20 or more	50	Future Electronics
Vehicle	1	1500	3D-Printed
Raspberry PI Camera	1	800	Future Electronics
IMU 6050 Sensor	1	650	Ampere Electronics
Breadboard	1	35	Ampere Electronics
L-298 H-bridge	1	70	Ampere Electronics

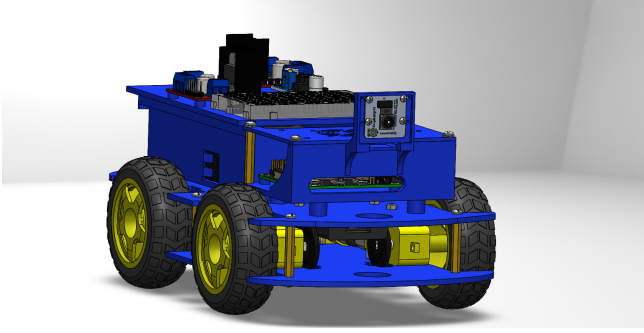


Figure 1: Solidworks Design



Figure 3: Picture of a QR Code

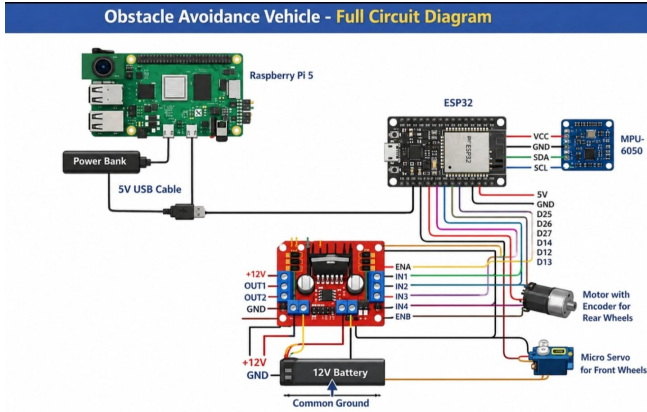


Figure 2: Electric circuit diagram

IV. ALGORITHM

A. Obstacle Detection Logic

A Raspberry Pi Camera Module 2 is used for obstacle detection. The obstacle is detected using a QR code that

is attached to it. QR codes like the one shown in figure 3 consist of black markers on a white background. They are characterized by 3 black squares that form an L shape.

The captured RGB frame is first transformed into a grayscale image so that histogram equalization can be done for lighting enhancing. After that the image is transformed into a binary image, where the three black squares that form the L shape of a QR reader are searched for inside the image, which signify the presence of a QR code.

On QR code detection, the distance between the camera and the QR code is calculated using the following formula:

$$d = \frac{W \cdot f}{w} \quad (1)$$

where d is the distance between the camera and the obstacle, W is the real width of the QR code, f is the focal length and w is the pixel width.

If the calculated distance is less than a certain threshold then that means that car is close to the obstacle.

B. Algorithm Logic

The car can move between two lanes whose positions are known. On obstacle detection, the car switches to the other lane. Two micro-controllers are used: Raspberry Pi 5 and ESP32. The ESP32 takes the heading angle from the IMU and the car's speed from the encoder and sends them to the Pi. At the same time, it takes the PWM signal and the steering angle from the Pi and sends it to the DC motor and servo motor respectively.

The Pi consists of 4 nodes which are:

- Camera node: This node takes camera frames and does the processing steps that were mentioned previously and publishes a boolean representing if the car is close to the obstacle or not.
- Obstacle Avoidance node: This node publishes the DC motor's PWM and the desired lane to which the car will go depending on the boolean published by the camera node.
- Localization node: This node takes the current heading angle and the current velocity from the Arduino and publishes the car's current X-position, Y-position, heading angle and speed. This is done using an Extended Kalman Filter (EKF).
- Controller node: This node takes the published outputs from the Obstacle Avoidance node and the Localization node. It then calculates the desired steering angle using a stanely controller. Finally, the steering angle and the DC motor's PWM are sent to the ESP32.

C. Controller Logic

The stanley controller is a non-linear controller that uses the car's geometry in order to calculate the required steering angle. It uses the following equation:

$$\theta = \theta_e + \tan^{-1}\left(\frac{k * y_e}{v + \epsilon}\right) \quad (2)$$

where θ_e is the heading error which is the difference between the desired heading angle (0) and the current heading angle, k is the stanely gain, y_e is the cross-track error which is the difference between the desired Y-position (depends on the desired lane) and the current Y-position, v is the current velocity and ϵ is a small value to avoid dividing by zero.

V. SIMULATION RESULTS

A. Camera Detection Simulation

Figures 4 and 5 show two different images captured by the camera. In each figure a green box is added around the QR code and the distance between the QR code and the camera is shown. In figure 4, the obstacle is close to the camera so a message is printed showing that. In figure 5, the obstacle is far from the camera so a message is printed showing that too. The camera's maximum detection range is 60 cm, while the threshold for being close to an obstacle is 50 cm. The car was left tracking a straight line with an obstacle at a

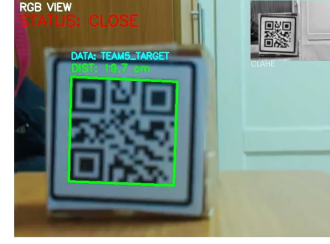


Figure 4: Obstacle Detection for a Close Obstacle

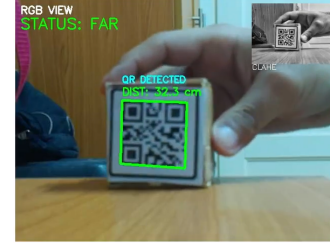


Figure 5: Obstacle Detection for a Far Obstacle

distance of 130 cm. The experiment was repeated several times, where the obstacle was always detected and the car performed the correct maneuver to avoid the obstacle. The average time the car took to detect the obstacle was found to be 0.342 s.

VI. CONCLUSION

In this project, a complete perception and control framework for obstacle detection and avoidance was successfully developed and implemented on a small-scale autonomous vehicle. By combining image processing techniques with real-time control algorithms, the system was able to reliably detect a QR-coded obstacle, estimate its distance, and execute appropriate avoidance maneuvers. The integration of localization through an Extended Kalman Filter and path tracking using a Stanley controller contributed to stable and accurate vehicle behavior. Experimental results confirmed the robustness and responsiveness of the system under different conditions. However, the current approach relies on QR codes for detection, which limits generalization to arbitrary obstacles. Future work could focus on incorporating more advanced computer vision or deep learning techniques to enable detection of diverse object types, as well as improving detection range. Overall, the project demonstrates a solid foundation for extending toward more sophisticated autonomous driving systems.

REFERENCES

- [1] S. Fernandes, D. Duseja, and R. Muthalagu, "Application of image processing techniques for autonomous cars," *Proceedings of Engineering and Technology Innovation*, vol. 17, p. 1, 2021.

- [2] X. Chen, Y. Liu, and K. Achuthan, "Wodis: Water obstacle detection network based on image segmentation for autonomous surface vehicles in maritime environments," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–13, 2021.
- [3] X. Zhao, H. Wu, Z. Xu, and H. Min, "Omni-directional obstacle detection for vehicles based on depth camera," *IEEE Access*, vol. 8, pp. 93 733–93 748, 2020.
- [4] T. Vojir, T. Šipka, R. Aljundi, N. Chumerin, D. O. Reino, and J. Matas, "Road anomaly detection by partial image reconstruction with segmentation coupling," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 15 651–15 660.
- [5] B. Hariprasanna and S. V. S. Devi, "Applying nature-inspired optimization: Ant and bee colony approaches for enhanced edge detection in image processing," in *2025 Third International Conference on Augmented Intelligence and Sustainable Systems (ICAISS)*. IEEE, 2025, pp. 160–165.